

Crazyflie 2.1 ROS 2 Humble Integration & Modular Control Nodes Document

Complete Setup, Node Architecture, and Battery Monitoring

Karan Kumar Anand

Department of Computer Science & Engineering
IIIT Delhi

July 27, 2026

Abstract

This document details the software architecture, modular node development, system configuration, and battery diagnostic tools implemented for the Crazyflie 2.1 micro-quadcopter within a ROS 2 Humble environment. It covers the `my_crazyflie` ROS 2 Python package containing specialized clients for arming, takeoff, landing, emergency stop, relative movement, and velocity control, as well as a standalone `cflib`-based battery monitoring utility.

Contents

1	Introduction & Workspace Overview	2
1.1	Environment Setup	2
2	ROS 2 Package Structure: <code>my_crazyflie</code>	2
2.1	Package Entry Points (<code>setup.py</code>)	2
3	Detailed Control Nodes Implementation	2
3.1	<code>arm.py</code> — Drone Arming Node	2
3.2	<code>takeoff.py</code> — Autonomous Takeoff Node	3
3.3	<code>land.py</code> — Controlled Landing Node	3
3.4	<code>emergency.py</code> — Emergency Stop Node	3
3.5	<code>sequence.py</code> — Complete Automated Flight Routine	3
3.6	<code>go_to.py</code> & <code>velocity_control.py</code>	3
4	Crazyflie Battery Diagnostics (<code>battery_check.py</code>)	3
4.1	Standalone Diagnostics Script	4
4.2	1S LiPo Battery Voltage Reference Table	4
4.3	Executing Battery Check	4
5	Command Summary & Verification Procedures	4
5.1	Step 1: Start Server (Terminal 1)	4
5.2	Step 2: Build Workspace & Run Flight Sequence (Terminal 2)	4
6	Conclusion	4

1 Introduction & Workspace Overview

The Crazyflie 2.1 workspace is organized under `~/cf_ws` operating on ROS 2 Humble Linux. The hardware platform used is a stock Crazyflie 2.1 quadcopter configured with zero expansion decks (0-deck setup). Due to the lack of optical flow or positioning decks, position estimation relies on open-loop or external positioning methods.

1.1 Environment Setup

Before running ROS 2 nodes or launching the Crazyflie server, the environment must be sourced:

```
1 cd ~/cf_ws
2 source /opt/ros/humble/setup.bash
3 source install/setup.bash
```

2 ROS 2 Package Structure: my_crazyflie

A custom ROS 2 Python package named `my_crazyflie` was constructed in `~/cf_ws/src/my_crazyflie`.

2.1 Package Entry Points (setup.py)

The console scripts registered in `setup.py` allow direct execution via `ros2 run my_crazyflie <script>`:

```
1 entry_points={
2     'console_scripts': [
3         'takeoff      = my_crazyflie.takeoff:main',
4         'land         = my_crazyflie.land:main',
5         'arm          = my_crazyflie.arm:main',
6         'emergency    = my_crazyflie.emergency:main',
7         'sequence     = my_crazyflie.sequence:main',
8         'go_to        = my_crazyflie.go_to:main',
9         'velocity_control = my_crazyflie.velocity_control:main',
10    ],
11 },
```

3 Detailed Control Nodes Implementation

3.1 arm.py — Drone Arming Node

In CrazySwarm2, arming is required before executing takeoff commands. This client uses `crazyflie_interfaces/srv/Arm`.

```
1 import rclpy
2 from rclpy.node import Node
3 from crazyflie_interfaces.srv import Arm
4
5 class ArmClient(Node):
6     def __init__(self):
7         super().__init__("arm_client")
8         self.client = self.create_client(Arm, "/cf231/arm")
9         while not self.client.wait_for_service(timeout_sec=1.0):
10             self.get_logger().info("Waiting for arm service...")
11
12     def arm(self):
13         request = Arm.Request()
```

```

14         request.arm = True
15         self.get_logger().info("Arming drone...")
16         future = self.client.call_async(request)
17         rclpy.spin_until_future_complete(self, future)
18         self.get_logger().info("Drone ARMED!")

```

3.2 takeoff.py — Autonomous Takeoff Node

Sends a height target and duration to /cf231/takeoff.

```

1 import rclpy
2 from rclpy.node import Node
3 from crazyflie_interfaces.srv import Takeoff
4
5 class TakeoffClient(Node):
6     def __init__(self):
7         super().__init__("takeoff_client")
8         self.client = self.create_client(Takeoff, "/cf231/takeoff")
9
10    def send_request(self):
11        request = Takeoff.Request()
12        request.group_mask = 0
13        request.height = 0.2
14        request.duration.sec = 2
15        future = self.client.call_async(request)
16        rclpy.spin_until_future_complete(self, future)

```

3.3 land.py — Controlled Landing Node

Commands the drone to smoothly descent to 0.0 m height over 2 seconds.

3.4 emergency.py — Emergency Stop Node

Calls /cf231/emergency (std.srvs/srv/Empty) to cut motor power immediately in case of unexpected flight drift.

3.5 sequence.py — Complete Automated Flight Routine

Executes the full safe flight state machine:

Arm → Takeoff (0.3m) → Hover (3s) → Land (0.0m)

3.6 go_to.py & velocity_control.py

- go_to.py: Uses crazyflie_interfaces/srv/GoTo to move to relative coordinates (x, y, z) .
- velocity_control.py: Publishes velocity and attitude commands directly over /cf231/cmd_vel_legacy

—

4 Crazyflie Battery Diagnostics (battery_check.py)

When using zero expansion decks, power management is critical. Low battery levels result in motor spin without sufficient thrust for lift-off.

4.1 Standalone Diagnostics Script

A Python script using the native `cflib` library was developed to poll battery status directly via Crazyradio without needing the ROS server active.

4.2 1S LiPo Battery Voltage Reference Table

The Crazyflie 2.1 uses a 1S 3.7V nominal LiPo battery (3.0 V cutoff to 4.2 V max).

Table 1: Crazyflie 2.1 LiPo 1S Voltage Classification

Voltage Range (V)	Approx. Charge (%)	Status Indicator	Recommended Action
≥ 4.10 V	$\sim 100\%$	Full	Ready for flight
≥ 3.90 V	$\sim 75\%$	Good	Safe for normal testing
≥ 3.75 V	$\sim 50\%$	Medium	Short flight tests only
≥ 3.50 V	$\sim 25\%$	Low	Recharge soon
≥ 3.20 V	$\sim 10\%$	Very Low	Land & recharge immediately
< 3.20 V	$< 5\%$	Critical	Do NOT operate (Battery protection)

4.3 Executing Battery Check

Ensure the Crazyflie ROS server is stopped, then run:

```
1 python3 src/my_crazyflie/my_crazyflie/battery_check.py
```

5 Command Summary & Verification Procedures

5.1 Step 1: Start Server (Terminal 1)

```
1 cd ~/cf_ws
2 source /opt/ros/humble/setup.bash
3 source install/setup.bash
4 ros2 launch crazyflie launch.py
```

5.2 Step 2: Build Workspace & Run Flight Sequence (Terminal 2)

```
1 cd ~/cf_ws
2 source /opt/ros/humble/setup.bash
3 colcon build --symlink-install
4 source install/setup.bash
5
6 # Run automated flight sequence
7 ros2 run my_crazyflie sequence
```

6 Conclusion

The integration of ROS 2 Humble with the Crazyflie 2.1 micro-quadcopter under `~/cf_ws` has been successfully verified. With arming interfaces, automated flight sequences, and battery diagnostic scripts implemented, the system is fully operational for autonomous aerial robotics research.